

ECSE 420 Assignment 3

April 21, 2026

Alisha Malik 261071602

Vivek Motta 260962299

Question 1

Question 1.1

L' is the largest array size or working-set size that still fits in cache, specifically $\text{cache_size} / \text{element_size}$ (cache size expressed in number of elements). When the array has fewer than L' elements, everything fits in cache. After the first pass loads it all in, every subsequent access is a hit, no matter how large the stride is, so the average time stays flat. t_0 is just the cache hit time, the cost of reading something that's already sitting in cache.

Question 1.2

t_1 is the cost of a cache miss, the time it takes to access main memory. When L exceeds L' , the array is too big to fit in cache fully, and once the stride gets large enough that every access lands on a fresh cache line, every single read goes to memory. t_1 is that penalty.

Question 1.3

Region 1: The whole array fits in cache. After the cold misses on the first pass, everything is loaded in and stays there. Stride doesn't matter at this point because no matter where in the array the benchmark is jumping, the data is already in cache. Time stays constant at t_0 .

Region 2: The array is now bigger than the cache. But for small strides, spatial locality helps; the cache line holds 4 words, so one miss brings in 4 consecutive elements. If the stride is less than 4, data within the same cache line gets reused. The effective miss rate is approximately $s/4$ and one miss fetches 4 words and with stride s you only reuse $4/s$ of them, so only $s/4$ of accesses actually miss. As the stride grows toward 4, more of each cache line gets wasted, the miss rate goes up, and average access time climbs from t_0 toward t_1 .

Region 3: Once the stride hits 4 words or more, every access touches a different cache line. No spatial locality, no reuse. Every read is a miss. The time flattens out at t_1 , making this the worst-case behavior.

Question 1.4

1.4: ALock pads its flag array so each thread's slot sits on its own cache line. The goal is to prevent false sharing without padding as one thread updating its flag invalidates the whole line, kicking out neighbouring threads' flags unnecessarily.

The problem is what padding does to the array. It inflates the stride between elements to a full cache line (4 words here), which is exactly the threshold where Region 3 begins, and blows up the total array size. With enough threads, the padded array can easily exceed L' .

That puts it right in Region 3 as stride equals cache line size, L exceeds L' . Every flag access becomes a guaranteed cache miss. Without padding, multiple flags could share a line, keeping the array smaller and potentially under L' , meaning accesses stay closer to t_0 .

So padding fixes false sharing but introduces a different problem. Instead of occasional unnecessary invalidations, there's a cache miss on every single lock acquisition. Under high contention with many threads spinning simultaneously, each spin generates a memory bus request which is the same bus traffic padding was originally meant to reduce by eliminating false sharing invalidations. This means padding can end up being worse than the problem it solved, and reduces overall lock performance.

Question 2

To compile and run Question 2:

1. *Navigate to the root of the project directory*

2. *Compile:*

```
javac -d . ca/mcgill/ecse420/a3/FineGrainedList.java
```

```
javac -d . ca/mcgill/ecse420/a3/FineGrainedListTest.java
```

3. *Run with assertions enabled:*

```
java -ea ca.mcgill.ecse420.a3.FineGrainedListTest
```

Expected output:

basic test passed

remove test passed

concurrent test passed

all tests passed

2.1

The contains() method follows the exact same hand over hand locking pattern as add() and remove(). It acquires head's lock before entering the outer try block, then immediately locks curr (the first real node) inside the inner try block. As the traversal advances, pred's lock is released only after curr.next is locked ensuring two locks are always held simultaneously. Once the loop exits (when curr.key >= key), the method returns curr.key == key without modifying the list.

2.2

Why the implementation is correct:

contains() uses the exact same hand-over-hand locking pattern as add() and remove(). At every point during traversal, two locks are held, pred and curr. Before releasing pred's lock, curr's lock is already acquired. This means no other thread can insert or remove a node between pred and curr while we're traversing, so we can't skip over or miss the target node.

The three tests cover the three cases that matter: basic correctness (is it actually finding what's there), concurrent safety (does it hold up when threads are adding simultaneously), and consistency with remove (does it reflect deletions properly).

While these tests do not prove correctness formally, they provide strong evidence that contains() behaves correctly under both sequential and concurrent scenarios.

```
C:\Users\16479\Desktop\A3>java -ea ca.mcgill.ecse420.a3.FineGrainedListTest
basic test passed
remove test passed
concurrent test passed
all tests passed
```

Question 3

To compile and run Question 3:

1. Navigate to the root of the project directory

2. Compile:

```
javac ca/mcgill/ecse420/a3/BoundedArrayQueue.java
```

3. Run:

```
java ca.mcgill.ecse420.a3.BoundedArrayQueue
```

Output:

Total produced : 200

Total consumed : 200

Queue empty : true

TEST PASSED

3.1

The array-backed bounded queue uses a circular `T[]` items array plus two independent `ReentrantLocks`, `tailLock` (guards `enq`) and `headLock` (guards `deq`), and an `AtomicInteger` `size`. Separate locks allow a concurrent enqueue and dequeue to proceed simultaneously as long as the queue is neither full nor empty. `enq(T x)` acquires `tailLock`, waits on `notFull` while `size == capacity`, writes `items[tail]`, advances `tail = (tail+1) % capacity` and calls `size.getAndIncrement()`. If the old size was 0, it briefly acquires `headLock` and calls `notEmpty.signalAll()` to wake blocked dequeuers. Symmetrically, `deq()` acquires `headLock`, waits on `notEmpty` while `size == 0`, reads `items[head]`, advances `head = (head+1) % capacity`, and calls `size.getAndDecrement()`. If the old size was capacity, it briefly acquires `tailLock` and calls `notFull.signalAll()` to wake blocked enqueueers. `signalAll()` is used throughout rather than `signal()` to prevent lost wakeups when multiple threads are waiting.

Correctness is guaranteed because `enq` and `deq` touch different fields (`tail` vs `head`), so they never conflict on data. The `AtomicInteger` ensures size changes are visible across both locks without requiring one lock to subsume the other. Cross-lock signalling only happens when the queue transitions from empty to non-empty or full to non-full, and critically the signal is always issued after releasing the first lock, ensuring the implementation never holds two locks simultaneously and avoiding deadlock.

3.2 .

An array based implementation offers several advantages over a linked list approach. **Cache locality:** array elements are contiguous in memory, so sequential access benefits from hardware prefetching and cache-line reuse, whereas a linked list

scatters nodes across the heap causing frequent cache misses. **No dynamic allocation:** once the array is allocated at construction, enq and deq never call new, eliminating per operation allocation overhead and GC pressure that linked list queues incur by allocating a new Node on every enqueue. **Lower per-element overhead:** each linked-list node carries an object header (16 bytes) and a next pointer (8 bytes), while an array slot is just a 4 to 8 byte reference. **Predictable memory footprint:** since capacity is fixed at construction time, the worst case memory usage is explicit and bounded. **No ABA problem:** array indices never drift the way recycled pointers can in lock-free linked lists, so the ABA problem does not arise.

3.3.

Transforming this algorithm to be lock-free presents several fundamental difficulties. **Multi-step updates** cannot be done atomically as each enq must both write items[tail] and advance tail, but there is no single CAS that covers both steps, meaning another thread can observe an inconsistent state between them. **No per-slot indirection** compounds this issue as the Michael-Scott lock-free queue hides its two phase commit inside a linked list pointer, but a circular array has no such indirection, so every slot would need its own atomic state flag such as an EMPTY/FULL sequence number, making the design far more complex. **Blocking on empty/full** is also hard to express without locks, since true lock-free code cannot use condition variables and implementing a non-blocking wait requires a separate CAS based waitlist such as a CLH queue per condition. **ABA on indices** introduces another hazard: if tail wraps all the way around the circular buffer, a stale CAS could falsely succeed, and the standard fix of using AtomicStampedReference per slot increases both memory and CAS width overhead. Finally, the **shared size counter serialises** all operations: the two-lock design shares one AtomicInteger size between both sides, and any lock-free design must eliminate this bottleneck, typically by using per-slot sequence numbers as in the Disruptor pattern, which is significantly harder to reason about and prove correct.

Question 4

To compile and run Question 4:

1. *Navigate to the root of the project directory*

2. *Compile:*

```
javac ca/mcgill/ecse420/a3/MatrixVectorMult.java
```

3. *Run:*

```
java ca.mcgill.ecse420.a3.MatrixVectorMult
```

Output:

ForkJoinPool parallelism: 8

4.1 Correctness Validation

3x3 known test:

Expected : [6.0, 15.0, 24.0]

Sequential: [6.0, 15.0, 24.0]

Parallel : [6.0, 15.0, 24.0]

Pass: true

Random 512x512: match=true

4.3 Benchmark n=4000

Thread pool parallelism (P): 8

Sequential avg: 14.2 ms

Threshold Par avg(ms) Speedup #Tasks(est)

4000	3.5	4.10	16000
2000	3.3	4.31	24000
1000	3.5	4.02	40000
500	3.8	3.71	72000
128	4.5	3.16	256000
64	7.6	1.86	504000
32	12.8	1.11	1008000

Optimal threshold: 2000 (speedup 4.31x)

Result correct at optimal threshold: true

4.4 Work and Critical-Path Analysis

$W(n) = \Theta(n^2)$: for $n=4000$, $W = 16000000$

$D(n) = \Theta(\log n)$: for $n=4000$, $D = 24$ steps

Parallelism = $W/D = \Theta(n^2/\log n)$: for $n=4000$, = 668573

Available $P=8$ threads; practical speedup bounded by $\min(P, \text{parallelism}) = P$

4.1

In terms of the correctness the double loop visits every (i,j) pair exactly once. After the inner loop for row i, $c[i] = \sum_j A[i][j]*x[j]$, which is exactly the definition of the i-th component of A times x. Validated against the known 3×3 case $A=[[1,2,3],[4,5,6],[7,8,9]]$, $x=[1,1,1] \rightarrow c=[6,15,24]$, and against random 512×512 inputs cross-checked with the parallel version.

4.2

The implementation uses a ForkJoinPool with RecursiveTask/RecursiveAction, which is the practical realisation of the Chapter 16 Future/thread-pool pattern. Unlike a blocking CachedThreadPool, fork() does not block the calling thread, so threads are never exhausted. The algorithm uses two levels of divide and conquer. At the first level, RowsTask splits the row range [rowLo, rowHi) in half, forks one half, computes the other in the current thread, and joins, so all rows ultimately run in parallel. At the second level, DotTask takes a single row i and splits the column range [lo, hi) in half, forks the left half, computes the right half in the current thread, then joins and adds the two results. The base case, reached when the range width is at or below THRESHOLD, computes the dot product sequentially.

Correctness follows by the same argument as the sequential version: every product $A[i][j]*x[j]$ is computed exactly once. The parallel reduction tree sums the same values in a different order, and floating point reassociativity is acceptable for non-exact arithmetic. Results are verified to agree with the sequential implementation within a relative tolerance of 1e-6.

4.3

Threshold	Par avg (ms)	Speedup	Tasks (estimated)
4000	3.5	4.10x	16000
2000	3.3	4.31x	24000

1000	3.5	4.02x	40000
500	3.8	3.71x	72000
128	4.5	3.16x	256000
64	7.6	1.86x	504000
32	12.8	1.11x	1008000

Sequential avg: 14.2 ms. Best: threshold = 2000, speedup $4.31\times$ with $P = 8$ threads.

The sweet spot lies between thresholds 4000 and 1000, where speedup stays above $4\times$. Below threshold 128, task creation overhead dominates and speedup collapses to near $1\times$. Efficiency of about 54% ($4.31\times$ out of a theoretical $8\times$) is explained by memory bandwidth saturation, since the 4000×4000 matrix occupies about 128 MB.

4.4

Work $W(n) = \theta(n^2)$

Each of the n^2 scalar products $A[i][j] * x[j]$ is computed exactly once across all rows. Fork/join overhead is $O(1)$ per call, and the total number of task calls is at most $2 \cdot (n/\text{thr}) \cdot n + 2 \cdot n = O(n^2/\text{thr})$, which is $O(n^2)$ for any constant threshold. Therefore $W(n) = \theta(n^2)$.

Critical Path $D(n) = \theta(\log n)$

The critical path has two components. First, RowsTask recursively splits all n rows into halves; at each level both halves run in parallel, so only one step per level lies on the critical path, giving a row-level depth of $\lceil \log_2 n \rceil = \theta(\log n)$. Second, each DotTask performs a binary reduction tree over the n columns of a single row, with depth $\lceil \log_2(n/\text{thr}) \rceil = \theta(\log n)$. Since these two phases lie on the same critical path, with one row's DotTask forming the bottleneck, they add to give $D(n) = \theta(\log n) + \theta(\log n) = \theta(\log n)$. For $n = 4000$ this gives $D \approx 2 \cdot \log_2(4000) \approx 24$ steps.

Parallelism $\Psi = W/D = \theta(n^2/\log n)$

For $n = 4000$, $\Psi \approx 16,000,000 / 24 \approx 668,000$. With $P = 8$ threads, since $\Psi \gg P$, the bottleneck is the number of available processors rather than the algorithm's inherent parallelism. In practice, speedup is bounded by $\min(P, \Psi) \approx P = 8$, and is further limited by memory bandwidth, since the 4000×4000 matrix occupies approximately 128 MB and stresses the memory bus, as well as task-scheduling overhead at fine granularities.

Appendix: Source Code

Question 2 Code

Q2.1) The following shows FineGrainedList.java (Q2.1), which implements the contains() method. See expected output above.

```
package ca.mcgill.ecse420.a3;

import java.util.concurrent.locks.ReentrantLock;

public class FineGrainedList<T> {

    private Node head;

    public FineGrainedList() {
        head = new Node(Integer.MIN_VALUE);
        head.next = new Node(Integer.MAX_VALUE);
    }

    public boolean add(T item) {
        int key = item.hashCode();
        head.lock();
        Node pred = head;
        try {
            Node curr = pred.next;
            curr.lock();
            try {
                while (curr.key < key) {
                    pred.unlock();
                    pred = curr;
                    curr = curr.next;
                    curr.lock();
                }
                if (curr.key == key) return false;
                Node node = new Node(item);
                node.next = curr;
                pred.next = node;
                return true;
            } finally {
                curr.unlock();
            }
        } finally {
            pred.unlock();
        }
    }
}
```



```
public boolean remove(T item) {
    int key = item.hashCode();
    head.lock();
    Node pred = head;
    try {
        Node curr = pred.next;
        curr.lock();
        try {
            while (curr.key < key) {
                pred.unlock();
                pred = curr;
                curr = curr.next;
                curr.lock();
            }
            if (curr.key == key) {
                pred.next = curr.next;
                return true;
            }
            return false;
        } finally {
            curr.unlock();
        }
    } finally {
        pred.unlock();
    }
}
```

```

public boolean contains(T item) {
    int key = item.hashCode();
    head.lock();
    Node pred = head;
    try {
        Node curr = pred.next;
        curr.lock();
        try {
            while (curr.key < key) {
                pred.unlock();
                pred = curr;
                curr = curr.next;
                curr.lock();
            }
            return curr.key == key;
        } finally {
            curr.unlock();
        }
    } finally {
        pred.unlock();
    }
}

```

```

private class Node {
    T item;
    int key;
    Node next;
    private final ReentrantLock lock = new ReentrantLock();

    Node(T item) {
        this.item = item;
        this.key = item.hashCode();
    }

    Node(int key) {
        this.key = key;
    }

    void lock() { lock.lock(); }
    void unlock() { lock.unlock(); }
}

```

Q2.2) The following shows FineGrainedListTest.java (Q2.2), which tests three scenarios: basic lookup, post-remove correctness, and concurrent adds.

```
package ca.mcgill.ecse420.a3;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class FineGrainedListTest {

    public static void main(String[] args) throws InterruptedException {
        FineGrainedList<Integer> list = new FineGrainedList<>();
        list.add(3);
        list.add(7);
        list.add(15);
        assert list.contains(3) : "should have 3";
        assert list.contains(7) : "should have 7";
        assert !list.contains(99) : "99 was never added";
        System.out.println("basic test passed");

        list.remove(7);
        assert !list.contains(7) : "7 was removed";
        assert list.contains(3) : "3 should still be there";
        System.out.println("remove test passed");

        FineGrainedList<Integer> concList = new FineGrainedList<>();
        ExecutorService pool = Executors.newFixedThreadPool(4);

        for (int t = 0; t < 4; t++) {
            final int id = t;
            pool.submit(() -> {
                for (int i = id * 10; i < id * 10 + 10; i++) {
                    concList.add(i);
                }
            });
        }
    }
}
```

```
pool.shutdown();  
pool.awaitTermination(5, TimeUnit.SECONDS);  
  
for (int i = 0; i < 40; i++) {  
    assert conclist.contains(i) : "missing " + i;  
}  
System.out.println("concurrent test passed");  
System.out.println("all tests passed");  
}  
}
```

Q3)

```

1 package ca.mcgill.ecse420.a3;
2
3 import java.util.concurrent.atomic.AtomicInteger;
4 import java.util.concurrent.locks.Condition;
5 import java.util.concurrent.locks.ReentrantLock;
6
7 //bounded lock-based queue backed by a circular array with separate head/tail locks
8 public class BoundedArrayQueue<T> {
9
10     private final T[] items; //circular storage array
11     private int head; //index of next item to dequeue
12     private int tail; //index of next slot to enqueue
13     private final int capacity;
14     private final AtomicInteger size; //current number of items
15
16     private final ReentrantLock headLock; //guards dequeue operations
17     private final Condition notEmpty; //dequeuers wait here
18
19     private final ReentrantLock tailLock; //guards enqueue operations
20     private final Condition notFull; //enqueueers wait here
21
22     public BoundedArrayQueue(int capacity) {
23         this.capacity = capacity;
24         this.items = (T[]) new Object[capacity];
25         this.head = 0;
26         this.tail = 0;
27         this.size = new AtomicInteger(0);
28
29         this.headLock = new ReentrantLock();
30         this.notEmpty = headLock.newCondition();
31
32         this.tailLock = new ReentrantLock();
33         this.notFull = tailLock.newCondition();
34     }
35
36     //blocking enqueue: acquires tailLock only, waits if queue is full
37     public void enq(T x) throws InterruptedException {
38         boolean mustWakeDequeuers = false;
39
40         tailLock.lock();
41         try {
42             while (size.get() == capacity) {
43                 notFull.await();
44             }
45             items[tail] = x;
46             tail = (tail + 1) % capacity;
47
48             //getAndIncrement returns old value; if 0, queue was empty so wake dequeuers
49             if (size.getAndIncrement() == 0) {
50                 mustWakeDequeuers = true;
51             }
52         } finally {
53             tailLock.unlock();
54         }
55

```

```

56         //signal after releasing tailLock to avoid holding two locks at once
57         if (mustWakeDequeuers) {
58             headLock.lock();
59             try {
60                 notEmpty.signalAll();
61             } finally {
62                 headLock.unlock();
63             }
64         }
65     }
66
67     //blocking dequeue: acquires headLock only, waits if queue is empty
68     public T deq() throws InterruptedException {
69         T result;
70         boolean mustWakeEnqueuers = false;
71
72         headLock.lock();
73         try {
74             while (size.get() == 0) {
75                 notEmpty.await();
76             }
77             result = items[head];
78             items[head] = null;
79             head = (head + 1) % capacity;
80
81             //getAndDecrement returns old value; if capacity, queue was full so wake enqueuers
82             if (size.getAndDecrement() == capacity) {
83                 mustWakeEnqueuers = true;
84             }
85         } finally {
86             headLock.unlock();
87         }
88
89         //signal after releasing headLock to avoid holding two locks at once
90         if (mustWakeEnqueuers) {
91             tailLock.lock();
92             try {
93                 notFull.signalAll();
94             } finally {
95                 tailLock.unlock();
96             }
97         }
98         return result;
99     }
100
101     public int size() { return size.get(); }
102     public boolean isEmpty() { return size.get() == 0; }
103     public boolean isFull() { return size.get() == capacity; }
104

```

```

105 //correctness test: 4 producers and 4 consumers, verifies all items are consumed
106 public static void main(String[] args) throws InterruptedException {
107     final int CAPACITY = 10;
108     final int PRODUCERS = 4;
109     final int CONSUMERS = 4;
110     final int ITEMS_PER_PRODUCER = 50;
111
112     BoundedArrayQueue<Integer> queue =
113         new BoundedArrayQueue<>(CAPACITY);
114
115     Thread[] producers = new Thread[PRODUCERS];
116     for (int p = 0; p < PRODUCERS; p++) {
117         final int pid = p;
118         producers[p] = new Thread(() -> {
119             for (int i = 0; i < ITEMS_PER_PRODUCER; i++) {
120                 try {
121                     queue.enq(pid * 1000 + i);
122                 } catch (InterruptedException e) {
123                     Thread.currentThread().interrupt();
124                 }
125             }
126         }, "Producer-" + p);
127     }
128
129     int[] consumed = new int[CONSUMERS];
130     Thread[] consumers = new Thread[CONSUMERS];
131     for (int c = 0; c < CONSUMERS; c++) {
132         final int cid = c;
133         consumers[c] = new Thread(() -> {
134             int count = 0;
135             while (count < ITEMS_PER_PRODUCER) {
136                 try {
137                     queue.deq();
138                     count++;
139                 } catch (InterruptedException e) {
140                     Thread.currentThread().interrupt();
141                 }
142             }
143             consumed[cid] = count;
144         }, "Consumer-" + c);
145     }
146
147     for (Thread c : consumers) c.start();
148     for (Thread p : producers) p.start();
149
150     for (Thread p : producers) p.join();
151     for (Thread c : consumers) c.join();
152
153     int totalConsumed = 0;
154     for (int c : consumed) totalConsumed += c;
155
156     System.out.println("Total produced : " + (PRODUCERS * ITEMS_PER_PRODUCER));
157     System.out.println("Total consumed : " + totalConsumed);
158     System.out.println("Queue empty : " + queue.isEmpty());
159     System.out.println("TEST " + (totalConsumed == PRODUCERS * ITEMS_PER_PRODUCER
160         && queue.isEmpty() ? "PASSED" : "FAILED"));
161 }
162 }

```

Q4)

```

1 package ca.mcgill.ecse420.a3;
2
3 import java.util.concurrent.*;
4 import java.util.*;
5
6 //parallel matrix-vector multiplication using ForkJoinPool
7 //sequential:  $O(n^2)$  work, parallel: work  $\Theta(n^2)$ , critical path  $\Theta(\log n)$ 
8 public class MatrixVectorMult {
9
10     static ForkJoinPool pool;
11     static int THRESHOLD = 64; //sub-ranges at or below this size are computed sequentially
12
13     //sequential matrix-vector multiply:  $c = A * x$ 
14     public static double[] seqMultiply(double[][] A, double[] x) {
15         int n = x.length;
16         double[] c = new double[n];
17         for (int i = 0; i < n; i++) {
18             double sum = 0.0;
19             for (int j = 0; j < n; j++) sum += A[i][j] * x[j];
20             c[i] = sum;
21         }
22         return c;
23     }
24
25     //recursive parallel dot product for a single row over columns [lo, hi)
26     static class DotTask extends RecursiveTask<Double> {
27         final double[][] A; final double[] x;
28         final int row, lo, hi;
29         DotTask(double[][] A, double[] x, int row, int lo, int hi) {
30             this.A=A; this.x=x; this.row=row; this.lo=lo; this.hi=hi;
31         }
32         protected Double compute() {
33             int w = hi - lo;
34             if (w <= THRESHOLD) {
35                 double s = 0;
36                 for (int j=lo; j<hi; j++) s += A[row][j] * x[j];
37                 return s;
38             }
39             int mid = lo + w/2;
40             DotTask left = new DotTask(A, x, row, lo, mid);
41             DotTask right = new DotTask(A, x, row, mid, hi);
42             left.fork();
43             double r = right.compute();
44             return left.join() + r;
45         }
46     }
47
48     //recursively fans out across rows [rowLo, rowHi), writing results into c[]
49     static class RowsTask extends RecursiveAction {
50         final double[][] A; final double[] x; final double[] c;
51         final int rowLo, rowHi;
52         RowsTask(double[][] A, double[] x, double[] c, int rowLo, int rowHi) {
53             this.A=A; this.x=x; this.c=c; this.rowLo=rowLo; this.rowHi=rowHi;
54         }
55         protected void compute() {
56             if (rowHi - rowLo == 1) {
57                 c[rowLo] = new DotTask(A, x, rowLo, 0, x.length).compute();
58                 return;
59             }
60             int mid = rowLo + (rowHi - rowLo) / 2;
61             RowsTask lo = new RowsTask(A, x, c, rowLo, mid);
62             RowsTask hi = new RowsTask(A, x, c, mid, rowHi);

```



```

63         lo.fork();
64         hi.compute();
65         lo.join();
66     }
67 }
68
69 //parallel matrix-vector multiply: submits all rows as concurrent ForkJoin tasks
70 public static double[] parMultiply(double[][] A, double[] x) {
71     int n = x.length;
72     double[] c = new double[n];
73     pool.invoke(new RowsTask(A, x, c, 0, n));
74     return c;
75 }
76
77 //returns true if arrays a and b agree within 1e-6 relative tolerance
78 static boolean approxEqual(double[] a, double[] b) {
79     for (int i=0; i<a.length; i++)
80         if (Math.abs(a[i]-b[i]) > 1e-6 * (Math.abs(a[i]) + 1)) return false;
81     return true;
82 }
83
84 static double[][] randomMatrix(int n, Random rng) {
85     double[][] A = new double[n][n];
86     for (int i=0; i<n; i++) for (int j=0; j<n; j++) A[i][j] = rng.nextDouble()*2 - 1;
87     return A;
88 }
89
90 static double[] randomVector(int n, Random rng) {
91     double[] x = new double[n];
92     for (int j=0; j<n; j++) x[j] = rng.nextDouble()*2 - 1;
93     return x;
94 }
95
96 //validates sequential and parallel results against a known 3x3 answer and a random 512x512 input
97 static void validateCorrectness() throws Exception {
98     System.out.println("4.1 Correctness Validation");
99
100     double[][] A3 = {{1,2,3},{4,5,6},{7,8,9}};
101     double[] x3 = {1,1,1}, exp3 = {6,15,24};
102     double[] s3 = seqMultiply(A3, x3);
103     int savedThr = THRESHOLD; THRESHOLD = 1;
104     double[] p3 = parMultiply(A3, x3);
105     THRESHOLD = savedThr;
106     System.out.println("3x3 known test:");
107     System.out.println("Expected : " + Arrays.toString(exp3));
108     System.out.println("Sequential: " + Arrays.toString(s3));
109     System.out.println("Parallel : " + Arrays.toString(p3));
110     System.out.println("Pass: " + (Arrays.equals(s3, exp3) && approxEqual(p3, exp3)));
111
112     Random rng = new Random(7);
113     int n = 512; double[][] B = randomMatrix(n, rng); double[] v = randomVector(n, rng);
114     double[] sB = seqMultiply(B, v);
115     THRESHOLD = 32; double[] pB = parMultiply(B, v); THRESHOLD = savedThr;
116     System.out.printf("Random %dx%d: match=%b\n", n, n, approxEqual(sB, pB));
117 }
118
119 //benchmarks sequential and parallel multiply for n=4000 across a range of threshold values
120 static void benchmark(int n) throws Exception {

```

```

120 static void benchmark(int n) throws Exception {
121     System.out.printf("4.3 Benchmark n=%d %n", n);
122     int P = pool.getParallelism();
123     System.out.println("Thread pool parallelism (P): " + P);
124
125     Random rng = new Random(42);
126     double[][] A = randomMatrix(n, rng); double[] x = randomVector(n, rng);
127
128     final int WARMUP = 3, RUNS = 5;
129
130     for (int r=0; r<WARMUP; r++) seqMultiply(A, x);
131     long seqNs = 0;
132     for (int r=0; r<RUNS; r++) { long t = System.nanoTime(); seqMultiply(A, x); seqNs += System.nanoTime() - t; }
133     double seqMs = seqNs / 1e6 / RUNS;
134     System.out.printf("Sequential avg: %.1f ms%n", seqMs);
135
136     int[] thresholds = { n, n/2, n/4, n/8, 128, 64, 32 };
137     System.out.printf("%-10s %-14s %-10s %-14s%n", "Threshold", "Par avg(ms)", "Speedup", "#Tasks(est)");
138     System.out.println("-".repeat(52));
139
140     int bestThr = n; double bestSpeedup = 0;
141     for (int thr : thresholds) {
142         if (thr < 1) thr = 1;
143         THRESHOLD = thr;
144         for (int r=0; r<WARMUP; r++) parMultiply(A, x);
145         long parNs = 0;
146         for (int r=0; r<RUNS; r++) { long t = System.nanoTime(); parMultiply(A, x); parNs += System.nanoTime() - t; }
147         double parMs = parNs / 1e6 / RUNS;
148         double speedup = seqMs / parMs;
149         long tasks = 2L * (n / thr) * n + 2L * n;
150         if (speedup > bestSpeedup) { bestSpeedup = speedup; bestThr = thr; }
151         System.out.printf("%-10d %-14.1f %-10.2f %-14d%n", thr, parMs, speedup, tasks);
152     }
153
154     THRESHOLD = bestThr;
155     double[] cSeq = seqMultiply(A, x), cPar = parMultiply(A, x);
156     System.out.printf("%nOptimal threshold: %d (speedup %.2fx)%n", bestThr, bestSpeedup);
157     System.out.printf("Result correct at optimal threshold: %b%n", approxEqual(cSeq, cPar));
158 }
159
160 //prints work, critical path, and parallelism analysis for n=4000
161 static void printAnalysis(int n) {
162     int P = pool.getParallelism();
163     double logN = Math.log(n) / Math.log(2);
164     System.out.println("\n4.4 Work and Critical-Path Analysis");
165     System.out.printf("W(n) = Theta(n^2): for n=%d, W = %d%n", n, (long)n*n);
166     System.out.printf("D(n) = Theta(log n): for n=%d, D = %.0f steps%n", n, 2*logN);
167     System.out.printf("Parallelism = W/D = Theta(n^2/log n): for n=%d, = %.0f%n", n, (double)n*n/(2*logN));
168     System.out.printf("Available P=%d threads; practical speedup bounded by min(P, parallelism) = P%n", P);
169 }
170
171 public static void main(String[] args) throws Exception {
172     int P = Runtime.getRuntime().availableProcessors();
173     pool = new ForkJoinPool(P);
174     System.out.println("ForkJoinPool parallelism: " + P + "\n");
175
176     validateCorrectness();
177
178     int n = 4000;
179     benchmark(n);
180     printAnalysis(n);

```

```

181
182     pool.shutdown();
183 }
184 }
185

```